

Relatório

Mini E-commerce Distribuído

1. Comunicação entre os microsserviços

A comunicação foi implementada por HTTP/REST. O cliente acessa apenas o API Gateway na porta 5000. O Gateway encaminha as chamadas para os serviços internos usando a biblioteca axios, preservando o método HTTP, o corpo JSON e o header Authorization. Os microsserviços são independentes e executam em portas separadas: 5001, 5002 e 5003.

Quando uma requisição exige autenticação, o Gateway repassa o token JWT recebido no header. Cada serviço valida o token localmente com a mesma chave secreta, JWT_SECRET, sem necessidade de consulta síncrona ao Users Service em cada operação.

No ambiente Docker Compose, a comunicação interna utiliza os nomes dos containers como hostnames:

- http://users:5001
- http://products:5002
- http://orders:5003

Essa abordagem isola o cliente dos detalhes de rede interna e centraliza o ponto de entrada na aplicação.

2. Estratégia de consistência na replicação

O Products Service utiliza duas réplicas JSON locais:

- products_replica_1.json
- products_replica_2.json

A estratégia adotada foi uma consistência forte simplificada. Toda escrita de produto é realizada nas duas réplicas antes de confirmar sucesso ao cliente. O sistema só retorna 201 depois que ambos os arquivos foram atualizados. As leituras, feitas por GET /products e GET /products/:id, são distribuídas por round-robin entre as réplicas, alternando entre replica_1 e replica_2 a cada requisição.

Como as escritas são síncronas nas duas réplicas, reduz-se a chance de leitura desatualizada no modelo simplificado adotado. A resposta das leituras indica de qual réplica os dados foram obtidos, facilitando a verificação do mecanismo.

Não se trata de uma replicação robusta de produção: as réplicas continuam sendo arquivos JSON locais, no mesmo serviço, e não nós independentes distribuídos. Porém, a implementação possui uma rotina simples de reconciliação automática entre as duas réplicas, que compara os arquivos, une os produtos pelo ID e corrige divergências básicas.

3. Falha no Serviço de Pedidos

Se o Orders Service cair, o Gateway detecta a falha por meio do heartbeat. A cada 5 segundos, o Gateway envia GET /health para cada microsserviço, com timeout de 2 segundos. Quando o serviço de pedidos deixa de responder, o contador de falhas é incrementado. Após 2 falhas consecutivas, o serviço é marcado como indisponível e um log é registrado com timestamp.

Enquanto indisponível, as rotas de pedidos, POST /orders e GET /orders/:userId, retornam a seguinte resposta com status HTTP 503 Service Unavailable:

```
{
  "error": "Serviço indisponível no momento"
}
```

Os serviços de usuários e produtos continuam funcionando normalmente, pois são microsserviços independentes. O Gateway bloqueia apenas as rotas associadas ao serviço indisponível.

Quando o Orders Service volta a responder, o Gateway registra a recuperação nos logs, zera o contador de falhas e libera as rotas novamente. O endpoint GET /status permite consultar o estado atual de cada serviço monitorado.

4. Segurança com JWT e controle de admin

O login no Users Service gera um JWT assinado com chave secreta, JWT_SECRET, com expiração de 1 hora. O token contém os campos userId, email, role e exp. A senha é armazenada com hash bcryptjs e nunca retornada nas respostas.

O Products Service valida o JWT antes de permitir criação de produto por POST /products. Além de verificar assinatura e expiração, verifica o campo role. Somente tokens com role = admin podem criar produtos. Se um usuário comum tenta criar produto, o serviço retorna erro de acesso negado com status 403 Forbidden.

No Users Service, usuário comum só consulta seus próprios dados, por GET /users/:id; admin acessa qualquer usuário. No Orders Service, a mesma regra se aplica à listagem de pedidos por GET /orders/:userId. Isso impede que usuários comuns executem ações administrativas, desde que a chave secreta do JWT permaneça protegida.

5. Limitações em relação a um sistema real

- Uso de arquivos JSON em vez de banco de dados transacional.
- Ausência de controle robusto de concorrência em escritas simultâneas.
- Replicação simples e local, sem réplicas em servidores distintos.
- Ausência de rollback se uma réplica falhar durante uma escrita.
- Ausência de balanceador de carga real entre instâncias.
- Ausência de HTTPS/TLS entre serviços.
- Ausência de autenticação mútua entre microsserviços.
- Ausência de fila de mensagens para comunicação assíncrona.
- Ausência de observabilidade avançada, como métricas e tracing distribuído.
- Logs simples apenas no terminal, sem agregação centralizada.
- Estoque do produto não é reduzido automaticamente ao criar pedido.
- Orders Service não valida se o produto existe antes de criar pedido.
- Variáveis sensíveis simplificadas para fins didáticos, como JWT_SECRET=supersecret.